Assignment 02

Code:-

List.java { Interface }

```
package in.cdac;
public interface List {
		public void addFirst(int data);
		public void addLast(int data);
		public int deleteFirst();
		public int deleteLast();
		public void displayList();
}
```

SinglyLinkedListADT.java { Class that implements List Interface }

```
package in.cdac;
public class SinglyLinkedListADT implements List {
	public class Node {
		int data;
		Node next;
	}
	private Node head;
	private Node tail;

	public SinglyLinkedListADT() {
		this.head = null;
		this.tail = null;
	}


	public boolean isEmpty() {
		if(head == null) {
			return true;
		}
		return false;
	}
	@Override
	public void addFirst(int data) {
```

```
        Node newNode = new Node();
        newNode.data = data;
        newNode.next = head;
        head = newNode;

        if (tail == null) {
                tail = head;
        }
}

@Override
public void addLast(int data) {
        Node newNode = new Node();
        newNode.data = data;
        newNode.next = null;

        if(isEmpty()) {
                head = newNode;
                tail = newNode;
        }
        tail.next = newNode;
        tail = newNode;
}

@Override
public int deleteFirst(){
        if(head == null) {
                throw new RuntimeException("The list is Empty. ");
        }
        Node deletedNode = head;
        head = head.next;
        if(isEmpty()) {
                tail = null;
        }
        return deletedNode.data;
}

@Override
public int deleteLast(){
        if(isEmpty()) {
```

```
                throw new RuntimeException("The List is Empty");
            }
            Node deletedNode = tail;
            Node tmpNode = head;
            while(tmpNode.next != tail) {
                tmpNode = tmpNode.next;
            }
            tail = tmpNode;
            tmpNode.next = null;

            return deletedNode.data;
        }

        public int getHead() {
            return head.data;
        }

        @Override
        public void displayList(){
            if(isEmpty()) {
                throw new RuntimeException("The List is Empty");
            }
            Node traversalNode = head;
            while(traversalNode != null) {
                System.out.print(traversalNode.data + " ");
                traversalNode = traversalNode.next;
            }
            System.out.println();
        }
}
```

SinglyLinkedListTest.java { Main Program }

```
package in.cdac;
import java.util.Scanner;

public class SinglyLinkedListTest {

        public static void main(String[] args) throws Exception{
            Scanner scanner = new Scanner(System.in);
```

```
int choice;

SinglyLinkedListADT Slist = new SinglyLinkedListADT();

do {
	System.out.println("1. Add at First\n"
	+ "2. Add at Last\n"
	+ "3. Delete First\n"
	+ "4. Delete Last\n"
	+ "5. Display LinkedList\n"
	+ "6. Exit");
	choice = scanner.nextInt();
	switch(choice) {
		case 1: {
			System.out.println("Enter a number to Add at First:
");
			Slist.addFirst(scanner.nextInt());
		}
		break;
		case 2: {
			System.out.println("Enter a number to Add at Last:
");
			Slist.addLast(scanner.nextInt());
		}
		break;
		case 3: {
			System.out.println("Deleted element: " +
Slist.deleteFirst());
		}
		break;
		case 4: {
			System.out.println("Deleted element: " +
Slist.deleteLast());
		}
		break;
		case 5: {
			Slist.displayList();
		}
		break;
		case 6: {
```

```
                    System.out.println("Exiting.......");
                    }
                    break;
                }
            }while(choice != 6);
            scanner.close();

        }

}
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
1
Enter a number to Add at First:
25
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
1
Enter a number to Add at First:
15
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
3
Deleted element: 15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
2
Enter a number to Add at Last:
35
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
5
25 35
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
4
```

```
Deleted element: 35
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Display LinkedList
6. Exit
6
Exiting.......
```

QueueWithLinkedListADT.java { Class that implements Queue Interface }

```
package in.cdac;

public class QueueWithLinkedListADT implements Queue{

	SinglyLinkedListADT queueObj = new SinglyLinkedListADT();

	@Override
	public void enqueue(int data) {
		queueObj.addLast(data);
	}

	@Override
	public int dequeue() {
		return queueObj.deleteFirst();
	}


	@Override
	public boolean isEmpty() {
		return queueObj.isEmpty();
	}

	@Override
	public void displayQueue() {
		queueObj.displayList();
	}

}
```

QueueWithLinkedListMain.java { Main Program }

```
package in.cdac;
import java.util.Scanner;

public class QueueWithLinkedListMain {

	public static void main(String[] args) {
		Scanner scanner = new Scanner(System.in);
```

```
        int choice;

        QueueWithLinkedListADT queueObj = new QueueWithLinkedListADT();

        do {
            System.out.println("1. Enqueue\n"
                                    + "2. Dequeue\n"
                                    + "3. isEmpty\n"
                                    + "4. Display Queue\n"
                                    + "5. Exit");
            choice = scanner.nextInt();
            switch(choice) {
                case 1: {
                            System.out.println("Enter a number to enqueue: ");
                            queueObj.enqueue(scanner.nextInt());
                        }
                        break;
                case 2: {
                            System.out.println(queueObj.dequeue());
                        }
                        break;
                case 3: {
                            System.out.println(queueObj.isEmpty());
                        }
                        break;
                case 4: {
                            queueObj.displayQueue();
                        }
                        break;
                case 5: {
                        System.out.println("Exiting.....");
                        }
                        break;
            }
        }while(choice != 5);

        scanner.close();
    }

}
```

```
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
1
Enter a number to enqueue:
15
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
1
Enter a number to enqueue:
25
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
1
Enter a number to enqueue:
35
```

```
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
2
15
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
4
25 35
1. Enqueue
2. Dequeue
3. isEmpty
4. Display Queue
5. Exit
```

StackWithLinkedListADT.java { Class that implements Stack Interface }

```
package in.cdac;
public class StackWithLinkedListADT implements Stack {

	SinglyLinkedListADT stackObjAdt = new SinglyLinkedListADT();

	@Override
	public void displayStack() {
		stackObjAdt.displayList();
	}

	@Override
	public boolean isEmpty() {
		return stackObjAdt.isEmpty();

	}

	@Override
	public int peek() {
		return stackObjAdt.getHead();

	}

	@Override
	public int pop(){
		int data = stackObjAdt.deleteFirst();
		return data;
	}

	@Override
	public void push(int data){
		stackObjAdt.addFirst(data);
	}

}
```

StackWithLinkedListMain.java { Main Program }

```
package in.cdac;

import java.util.Scanner;

public class StackWithLinkedListMain {

	public static void main(String[] args) {

		Scanner scanner = new Scanner(System.in);
		int choice;

		StackWithLinkedListADT stackObj = new StackWithLinkedListADT();

		do {
			System.out.println("1. Push\n"
								+ "2. Pop\n"
								+ "3. Peek\n"
								+ "4. isEmpty\n"
								+ "5. Display Stack\n"
								+ "6. Exit");
			choice = scanner.nextInt();
			switch(choice) {
				case 1: {
						System.out.println("Enter a number to enqueue: ");
						stackObj.push(scanner.nextInt());
					}
					break;
				case 2: {
						System.out.println(stackObj.pop());
					}
					break;
				case 3: {
						System.out.println(stackObj.peek());
					}
					break;
				case 4: {
						System.out.println(stackObj.isEmpty());
					}
					break;
```

```
                    case 5: {
                                stackObj.displayStack();
                            }
                            break;
                    case 6: {
                            System.out.println("Exiting.....");
                            }
                            break;
                }
            }while(choice != 6);
            scanner.close();
        }
}
```

```
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
1
Enter a number to enqueue:
15
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
1
Enter a number to enqueue:
25
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
1
```

```
Enter a number to enqueue:
35
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
1
Enter a number to enqueue:
45
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
2
45
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
5
```

```
35 25 15
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
3
35
1. Push
2. Pop
3. Peek
4. isEmpty
5. Display Stack
6. Exit
```